INDIANA UNIVERSITY INDIANAPOLIS
School of Science • Department of Computer Science

MARO

Programming Language

Project Report • Version 1.0

Course	CSCI-C 212
Project	Maro — A Programming Language Built from Scratch
Language	Java 17+ (tree-walk interpreted)
Version	1.0
Members	Mario Espinoza-Gutierrez
Submitted	April 2026
Creater	Mario Espinoza-Gutierrez

This report provides a comprehensive overview of the Maro programming language project, including its architecture, design decisions, implementation details, member contributions, example programs, and future roadmap.

1. Executive Summary

Maro is a complete, fully functional programming language implemented from scratch in Java 17. The project was undertaken as a group assignment to demonstrate a deep understanding of how real programming languages are designed, parsed, and executed. Rather than using any external language-generation frameworks, the team hand-built every component , from the character-level lexer through to the tree-walk interpreter that runs Maro programs.

Sure there are still bugs to be fix in Maro, however the result is a clean, readable, dynamically-typed language with Python-inspired syntax. Maro supports variables, user-defined functions, closures, recursion, first-class functions, control flow (if/else, while, for-in), lists, maps, strings, booleans, null values, and a library of 21 built-in utility functions covering math, collections, and string manipulation.

Key Statistics

9 Java source files • 2,473 lines of annotated code • 4 pipeline stages • 21 built-in functions • 7 data types • 3 showcase programs

The entire codebase is written in Java. This makes the project useful both as a working language implementation and as a learning resource for students studying compilers and interpreters.

2. Project Overview

2.1 What is Maro?

Maro is a dynamically-typed, interpreted programming language. 'Dynamically typed' means variables do not require type declarations , at runtime, any variable can hold any value (a number, a string, a list, a function, etc.). 'Interpreted' means Maro programs are not compiled to machine code; instead, the interpreter reads the Abstract Syntax Tree (AST) produced by the parser and executes it directly in a process called tree-walk interpretation.

Maro's syntax draws heavily from Python and JavaScript while remaining its own language. It uses the let keyword for variable declarations, fun for functions, and

English words (and, or, not) for logical operators. Semicolons are optional , a deliberate design decision to reduce visual clutter.

2.2 Design Goals

- Simplicity — a minimal, clean syntax that is easy to read and write from day one
- Completeness — full support for the features needed for real programs: functions, closures, recursion, and data structures
- Portability — pure Java with zero external runtime dependencies; any system with Java 17+ can run Maro programs

2.3 Quick-Look Syntax

The following code block shows a complete Maro program, illustrating all major language features:

```
# Variables
let name = "Maro"
let version = 1

# Functions and closures
fun makeCounter(start) {
  let count = start
  fun next() {
    count = count + 1
    return count
  }
  return next
}

# Lists, for-in loop, and built-ins
let nums = [1, 2, 3, 4, 5]
for (n in nums) {
  print(str(n * 2))
}
```

3. System Architecture

3.1 The Four-Stage Pipeline

Maro's implementation follows the classic four-stage language pipeline found in real-world interpreters and compilers. Each stage transforms the program representation into something the next stage can work with:

Stage	Class	Input	Output	Responsibility
1	Lexer.java	Source text (String)	List<Token>	Scans characters, classifies tokens
2	Parser.java	List<Token>	Node.Program (AST)	Builds the syntax tree via grammar rules
3	Node.java	— (data structure)	— (data structure)	Defines all 22 AST node types
4	Interpreter.java	Node.Program (AST)	Side-effects / output	Walks the tree and executes each node

3.2 Source File Breakdown

The project consists of exactly nine Java source files, organised into four packages:

File	Lines	Description
maro/Maro.java	222	Entry point. Chains all four stages together. Contains a REPL and file-runner mode.
maro/lexer/TokenType.java	117	Enum listing all 37 token categories with inline documentation for each.
maro/lexer/Token.java	54	Immutable data class holding a token's type, text value, and source line number.
maro/lexer/Lexer.java	352	Character-level scanner that produces a List<Token> from raw source text.

maro/ast/Node.java	187	Sealed interface + 22 record types covering every statement and expression.
maro/parser/Parser.java	623	Recursive-descent parser implementing full operator-precedence climbing.
maro/interpreter/Environment.java	122	Linked chain of HashMaps representing variable scopes.
maro/interpreter/MaroFunction.java	74	Data class holding a user-defined function's params, body, and captured closure.
maro/interpreter/Interpreter.java	722	Tree-walk executor with execute() for statements and evaluate() for expressions.

3.3 Key Design Decisions

Sealed Interface for AST Nodes

Java 17's sealed interface feature was used for the Node type. This means only the 22 explicitly listed record types can implement Node. The benefit is exhaustive pattern matching , the compiler warns if any node type is unhandled in a switch expression, preventing whole classes of runtime bugs.

ReturnSignal Exception Pattern

When the interpreter encounters a return statement, it throws a private ReturnSignal exception (a RuntimeException subclass with stack-trace generation disabled for performance). This exception bubbles up through all nested execute() calls until it is caught by the function-call handler. This is a standard and elegant interpreter technique , it avoids threading a return value through every layer of the call stack.

Closure via Environment Chaining

Every Environment holds a reference to its parent scope. When a function is defined, the interpreter wraps it in a MaroFunction that captures a reference to the current environment (the closure). When the function is later called, a new child Environment is created with the captured environment as its parent , so the function still sees all variables from where it was defined, not where it was called.

Dynamic Typing via Java Object

Because Maro is dynamically typed, every value is represented as a plain Java Object at runtime. Type dispatch is done with instanceof checks and switch expressions. All numbers are stored as Java double (64-bit floating point), which handles both integers and decimals without needing separate types.

4. Use Cases

Maro was designed with three primary use cases in mind:

4.1 Language Design Experimentation

Because the pipeline is cleanly separated into four files, adding new features to Maro is straightforward. A student wanting to add a new keyword adds it to TokenType.java, handles it in the Lexer, creates a Node record in Node.java, parses it in Parser.java, and executes it in Interpreter.java. This makes Maro an ideal sandbox for exploring language design ideas such as new syntax forms, operator overloading, or pattern matching.

4.2 Scripting and Automation

Maro's clean Python-like syntax makes it suitable as a lightweight scripting language for small automation tasks, data processing scripts, and algorithmic experiments. The built-in library covers the most common needs: mathematical operations (sqrt, pow, floor, ceil, abs, max, min), string manipulation (upper, lower, split, join), and list/map operations (push, pop, keys, contains, range).

4.3 Compiler and PL Research Prototype

The tree-walk interpreter is the simplest execution model, but the clean AST makes Maro a natural starting point for more advanced research. Adding a static type checker requires only a new pass over the AST. Compiling to JVM bytecode using the ASM library is achievable by replacing the interpreter with a code-generator. The project therefore serves as a foundation for further study in programming language implementation.

5. Group Members

The project was completed by just me as a solo team. Each role, I took ownership of specific components while collaborating on overall design decisions.

Name	Role	Primary Files	%	Key Contributions
Mario	Lead Developer & Language	Maro.java, Lexer.java, TokenType.java,	**35%**	Language architecture, full

	Designer	Token.java		Lexer, REPL, entry point
Mario	Interpreter & Runtime Developer	Interpreter.java, Environment.java, MaroFunction.java	**30%**	Tree-walk interpreter, scopes, 21 built-in functions
Mario	AST & Parser Developer	Node.java, Parser.java	**20%**	All AST nodes (sealed interface), recursive-descent parser
Mario	Showcase & Documentation Lead	errors_showcase.maro, math_showcase.maro, README.md	**15%**	Three showcase programs, full reference documentation, presentation

5.1 Detailed Contributions

Lead Developer

- Designed the overall language architecture and the four-stage pipeline split
- Implemented the complete Lexer, handling all 37 token types including string escapes, negative numbers, and two-character operators (==, !=, <=, >=, ->)
- Wrote Maro.java, the entry point that chains the pipeline and provides both REPL and file-runner modes
- Led all major syntax design decisions: optional semicolons, English logical operators (and, or, not), # comments, and the let / fun keywords

Interpreter & Runtime

- Built the entire tree-walk interpreter (722 lines) — the largest single file in the project
- Implemented the linked Environment scope chain with get(), define(), and set() methods that correctly walk up to parent scopes
- Designed the MaroFunction data class and the closure-capture mechanism
- Implemented the ReturnSignal exception pattern for correct return-statement semantics
- Coded all 21 built-in functions including range(), push(), pop(), split(), join(), sqrt(), pow(), and type()

AST & Parser

- Designed all 22 AST node types in Node.java using Java's sealed interface and record features
- Implemented the full recursive-descent Parser (623 lines) with correct operator-precedence climbing across 9 levels
- Handled all grammar constructs: let, fun, if/else, while, for-in, print, return, function calls, indexing, field access, list/map literals

Showcase & Documentation

- Wrote errors_showcase.maro — seven sections covering division-by-zero protection, null safety, list bounds checking, type validation, loop guards, truthy/falsy edge cases, and nested collections
- Wrote math_showcase.maro — eleven sections covering arithmetic, statistics, Fibonacci (recursive and iterative), factorials, prime detection, GCD/LCM, FizzBuzz, bubble sort, binary search, triangle numbers, and Pascal's Triangle
- Produced the full Word reference document (maro_language.docx) with eight sections of documentation
- Designed both presentations (maro_presentation.pptx) with screenshot slides, member cards, and contribution breakdowns

6. Language Reference

6.1 Data Types

Type	Example	Notes
number	`42  3.14  -7`	64-bit double. Displayed as integer when whole (42, not 42.0).
string	`"hello"  ""`	Immutable. Supports + concatenation, * repetition, and index access.
bool	`true  false`	Returned by comparison and logical operators.
null	`null`	Represents the absence of a value. Evaluates as falsy.
list	`[1, 2, 3]`	Mutable, ordered, zero-indexed. Access with [i]. Supports negative indices.
map	`{name: "Rio"}`	Mutable key-value store. Access with .field or ["key"].
function	`fun f() { ... }`	First-class. Can be stored, passed, and returned.

6.2 Operators

Category	Operators	Notes
Arithmetic	+ - * / %	+ also concatenates strings; * repeats strings ("ha" * 3 → "hahaha")
Comparison	== != < > <= >=	Produces a bool. == works on any type via .equals().
Logical	and or not	Short-circuit: and/or stop evaluating once result is determined.
Assignment	=	Assigns to an existing variable. Use let to declare a new one.

6.3 Built-In Functions

Function	Returns	Description
len(x)	number	Length of a string, list, or map.
str(x)	string	Converts any value to its string form.
num(x)	number	Parses a string as a number.
type(x)	string	Returns the type name: "number", "string", "bool", "list", "map", "function", or "null".
range(n)	list	Returns [0, 1, ..., n-1].
range(start, end)	list	Returns [start, ..., end-1].
push(list, item)	list	Appends item to end of list. Mutates in place.
pop(list)	any	Removes and returns the last item.
keys(map)	list	Returns a list of all keys in the map.
contains(col, item)	bool	True if item exists in the list, map, or string.
split(str, sep)	list	Splits string on separator.
join(list, sep)	string	Joins list elements with separator string.
upper(str)	string	Converts string to UPPERCASE.
lower(str)	string	Converts string to lowercase.
abs(n)	number	Absolute value.
floor(n)	number	Rounds down to nearest integer.

ceil(n)	number	Rounds up to nearest integer.
sqrt(n)	number	Square root.
pow(base, exp)	number	Raises base to the power of exp.
max(a, b)	number	The larger of a and b.
min(a, b)	number	The smaller of a and b.

7. Showcase Programs

Three showcase programs were written to demonstrate the breadth of what Maro can do. Each is a standalone .maro file that can be run with the Maro interpreter.

7.1 showcase.maro — General Showcase

The main showcase file covers the full language surface in one program. It demonstrates variables of every type, function declarations, if/else branching, while and for-in loops, list and map creation, recursive functions (Fibonacci and factorial), closures and higher-order functions, string operations, and FizzBuzz. All are separate files into functionsShowcase, printShowcase, variablesShowcase , .maro files.

Sections covered

Hello World • Variables • Functions • If/Else • While loop • Lists • For-in loop • Maps • Factorial • Fibonacci • Higher-order functions • String operations • Closures • FizzBuzz

7.2 mathShowcase.maro — Mathematics

This file is dedicated to numeric computation. It demonstrates how Maro handles a wide range of mathematical algorithms, validating both the correctness of the arithmetic and the usability of built-in math functions.

Section	What it demonstrates
Basic Arithmetic	All operators (+, -, *, /, %) with pow() and sqrt()
Built-in Math	abs(), floor(), ceil() applied to a list of decimals

Statistics	sumList(), average(), findMax(), findMin() — all user-defined
Fibonacci	Both recursive and iterative implementations, up to fib(30)
Factorial & nCr	Recursive factorial, combinations(n, r) formula
Prime Numbers	Sieve using sqrt shortcut, finding all primes up to 60
GCD & LCM	Euclid's algorithm for GCD, LCM derived from GCD
FizzBuzz	Classic modulo-based solution for 1 to 30
Sorting	Bubble sort implementation + binary search on sorted list
Triangle Numbers	Formula k*(k+1)/2 for the first 10 triangle numbers
Pascal's Triangle	Iterative row-building algorithm for 6 rows

7.3 errorsShowcase.maro — Error Handling

This file demonstrates Maro's approach to safe programming. Because Maro does not yet have a try/catch syntax, all error handling is done via guard clauses — checking conditions before performing potentially dangerous operations and returning descriptive error messages instead of crashing.

Section	Guard pattern demonstrated
Division by zero	Check divisor == 0 before dividing; return error string
Null safety	Check name == null and len(name) == 0 before using
List bounds	Check 0 <= index < len(list) before accessing
Type validation	Check type(x) == "number" before arithmetic
Infinite loop guard	Check target <= 1000 before entering a counting loop
String edge cases	Empty strings, mixed-type concatenation, repetition
Truthy/falsy values	showTruthiness() helper tests 0, "", [], null, false, etc.
Scope edge cases	Demonstrates variable shadowing and closure behaviour
Recursion depth guard	Safe recursive sum with depth parameter and limit check
Nested collections	Matrix indexing, mixed-type lists, nested map access

8. Presentation Overview

The group presentation (maro_group_presentation.pptx) is an 11-slide deck designed to accompany a live spoken presentation of the project. It was built using Linux Libera Writer since I used Linux as my main laptop.

#	Slide Title	Content Summary
1	**Cover**	Title, course badge, tagline, IUI footer, decorative circles
2	**Project Overview**	What Maro does; 4 stat badges; pipeline diagram; 4 key-feature bullets
3	**Use Cases**	Four card layout: Education, Experimentation, Scripting, PL Research
4	**Group Members**	Four member cards with avatar circle, colour-coded role, and skill list
5	**Screenshot — Variables & Fns**	Syntax-highlighted hello.maro screenshot + 3 callout annotation cards
6	**Screenshot — Math Showcase**	math_showcase.maro with Fibonacci/prime code + callout cards
7	**Screenshot — Error Handling**	errors_showcase.maro with guard clause code + red error output
8	**Screenshot — REPL & Pipeline**	Interactive REPL session + 4-stage pipeline diagram side by side
9	**Member Contributions**	4 cards each with name, % progress bar, file list, 3 bullet contributions
10	**Conclusion**	What we achieved (6 bullets) + Next Steps panel (5 future features)
11	**Thank You**	Closing slide with member name, university, and keyword tag strip

9. How to Run Maro

9.1 Requirements

- Java 17 or later (uses sealed interfaces and record types)
- Any operating system — Windows, macOS, or Linux
- No external libraries or dependencies

9.2 Compile

From the project root directory, compile all nine source files into an out/ directory:

```
javac -d out $(find src/main/java/maro -name "*.java")
```

9.3 Run a File

Pass the path to any .maro file as a command-line argument:

```
java -cp out maro.Maro examples/showcase.maro
java -cp out maro.Maro examples/math_showcase.maro
java -cp out maro.Maro examples/errors_showcase.maro
```

9.4 Interactive REPL

Launch with no arguments to enter the Read-Eval-Print Loop. Type exit to quit:

```
java -cp out maro.Maro


==================================
 Maro Language v1.0
 Type 'exit' to quit
==================================
>>> let x = 10
>>> print(str(x * x))
100
>>> exit
```

Tip — Windows users

On Windows, replace the $(...) subshell with an explicit file list, or use: for /r src %f in (*.java) do javac -d out "%f"

WARNING

The REPL have some bugs and errors yet to be fixed.

10. Future Roadmap

The following features are planned for future versions of Maro, organised by estimated implementation effort:

Phase 1 — Short-Term (Easy)

- String interpolation — embed expressions directly in strings: "Hello, {name}!"
- Augmented assignment operators — x += 1, x -= 1, x *= 2, x /= 2
- Multiple return values — let a, b = someFunction()
- Trailing else-less if — if (cond) { singleStatement }

Phase 2 — Intermediate

- Classes and objects — class Point { let x; let y; fun distance(other) { ... } }
- Error handling — try { ... } catch (e) { ... } syntax with a MaroError type
- Module system — import "utils.maro" to split code across files
- Switch/match expression — match (x) { case 1: ... case 2: ... default: ... }

Phase 3 — Advanced

- Static type checker — a separate AST pass that reports type errors before runtime
- Bytecode compiler — emit JVM .class files using the ASM library for faster execution
- Standard library — file I/O, networking, date/time, and regular expressions
- Package manager — a simple way to share and import Maro libraries

11. Conclusion

Building Maro from scratch gave me a genuine, hands-on understanding of how real programming languages work at every level , not as abstract concepts in a textbook, but as actual, running Java code. Every decision had consequences. Choosing a flat token list over a structured token type forced the parser to use lookahead. Choosing dynamic typing over static typing made the interpreter simpler but required runtime type checks. Choosing tree-walk interpretation over bytecode compilation made the code readable but slower. These trade-offs are exactly what language designers and compiler engineers face in practice, and working through them in a real implementation made the lessons stick in a way that no amount of reading alone could achieve. The project delivered a complete, almost working, and well programming language in nine Java files in just 1 month. It can run real programs, supports closures and recursion, and includes a library of 21 built-in functions. Three showcase programs demonstrate its capabilities across general programming, mathematics, and error handling.

Project Summary

Maro v1.0 | 9 Java files | 2,473 lines | 4 pipeline stages | 21 built-ins | 7 data types | 3 showcase programs | Indiana University Indianapolis

Mario Espinoza-Gutierrez
Indiana University Indianapolis • April 2026